



Licence MIAGE

Rapport de projet Open Data

Optimisation d'itinéraires Vélib' en Île-de-France par la théorie des graphes

Projet réalisé de Avril 2026 à Mai 2026

Code source disponible sur GitHub :
<https://github.com/ornelamt/ProjetGrapheOpenData>

Membre

Mothu Ornela

Table des matières

1	Introduction	4
1.1	Contexte	4
1.2	Problématique	4
1.3	Objectifs	4
2	Données utilisées	4
2.1	Sources de données	4
2.1.1	API Vélib' temps réel	5
2.1.2	OSMnx et OpenStreetMap	5
2.1.3	Aménagements cyclables IDF	5
2.2	Nettoyage et traitement des données	5
2.2.1	Données Vélib'	5
2.2.2	Réseau cyclable OSMnx	6
2.2.3	Difficulté rencontrée	6
3	Modélisation en graphe	6
3.1	Construction du graphe	6
3.2	Caractéristiques du graphe	6
3.3	Visualisation avec Gephi	6
4	Algorithmes appliqués	7
4.1	Analyse de centralité	7
4.2	Composantes connexes	7
4.3	Algorithme de Dijkstra	8
4.3.1	Principe	8
4.3.2	Deux modes d'itinéraire	8
4.3.3	Processus complet de calcul	8
4.3.4	Identification des tronçons manquants	8
5	Résultats et analyse	8
5.1	Exemple de trajet	8
5.2	Tronçons manquants et coût de sécurisation	9
6	Interface web interactive	9
6.1	Architecture technique	9
6.2	Fonctionnalités principales	9
7	Difficultés rencontrées	10
8	Limites et perspectives	10
8.1	Limites identifiées	10
8.2	Perspectives d'amélioration	11
9	Conclusion	11
10	Bibliographie	13
11	Webographie	14

A Annexe : Code Python — Calcul du chemin sécurisé**15**

1 Introduction

Ce projet s'inscrit dans le cadre du cours de théorie des graphes. Il porte sur l'analyse et l'optimisation du réseau de vélos en libre-service **Vélib'** en Île-de-France. L'objectif est de modéliser ce réseau sous forme de graphe mathématique afin d'y appliquer des algorithmes classiques de la théorie des graphes et de développer un outil concret d'aide à la décision pour les cyclistes parisiens.

Le code source complet est disponible sur GitHub : <https://github.com/ornelamt/ProjetGrapheOpenData>

1.1 Contexte

Vélib' est le système de vélos en libre-service de la région Île-de-France, géré par Smovengo depuis 2018. Il compte aujourd'hui plus de 1 500 stations réparties sur Paris et sa petite couronne, proposant à la fois des vélos mécaniques et électriques.

Malgré l'essor du vélo comme mode de transport urbain, les cyclistes parisiens font face à un choix quotidien : emprunter le trajet le plus rapide, qui passe souvent par des routes partagées avec les voitures, ou choisir un trajet plus long mais sécurisé, en privilégiant les pistes cyclables. Cette problématique a guidé l'ensemble du projet.

1.2 Problématique

■ *Comment la théorie des graphes peut-elle aider un cycliste parisien à choisir entre rapidité et sécurité, en tenant compte des infrastructures cyclables disponibles et de la disponibilité en temps réel des stations Vélib' ?* ■

1.3 Objectifs

1. Modéliser le réseau Vélib' sous forme de graphe
2. Analyser la structure du réseau (centralité, composantes connexes)
3. Calculer des itinéraires optimaux via l'algorithme de Dijkstra
4. Identifier les tronçons cyclables manquants et estimer leur coût
5. Développer une interface web interactive avec données en temps réel

2 Données utilisées

2.1 Sources de données

Toutes les données utilisées dans ce projet sont **gratuites et publiques** — c'est ce qu'on appelle l'Open Data.

Source	Contenu	URL
API Vélib' temps réel	1 511 stations, vélos disponibles, bornettes libres	opendata.paris.fr
OSMnx / OpenStreetMap	Réseau cyclable et routier de Paris + petite couronne	openstreetmap.org
Aménagements cyclables IDF	146 103 tronçons de pistes cyclables IDF	data.iledefrance-mobilites.fr

TABLE 1 – Sources de données utilisées

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
1	GeoShape	osm_id	nom_com	nom_voie	insee_com	highway	sens_voie	revetement	panneaux	ad	ag	rv	longueur	notes	petite_ech	moyenn_ech	geo_point_2d	
2	'coordinates'	519634995.0	18e Arrondissement		75118	service	DOUBLE					limite 30	109		3	32	48.89922903163066, 2.3	
3	'coordinates'	1327637020.0	18e Arrondissement	Avenue de la F	75118	cyclway	NC			piste uni	piste uni	hors voirie		7	1	11	48.89965879996426, 2.3	
4	'coordinates'	648288280.0	18e Arrondissement	Avenue de la F	75118	cyclway	NC			piste uni	piste uni	hors voirie		59	1	11	48.901152549309934, 2.3	
5	'coordinates'	320826376.0	18e Arrondissement	Avenue de la F	75118	tertiary	DOUBLE	asphalt				limite 30	203		3	32	48.90054004379811, 2.35	
6	'coordinates'	648288652.0	Saint-Ouen	Rue des Poiss	93070	cyclway	NC	asphalt		piste uni	piste uni	hors voirie	109		1	11	48.90136811124878, 2.35	
7	'coordinates'	88502959.0	Saint-Ouen	Passage Jean	93070	living_street	UNIQUE	asphalt				20	52		3	33	48.903800824736056, 2.3	
8	'coordinates'	429670712.0	Saint-Ouen		93070	footway	NC	asphalt		autre chemin	autre chemin	hors voirie	30		4	41	48.90273106256494, 2.35	
9	'coordinates'	672733294.0	Saint-Ouen	Rue Debain	93070	residential	UNIQUE	asphalt			DSC	limite 30	79		3	32	48.90270322071204, 2.34	
10	'coordinates'	837203565.0	Saint-Ouen	Rue Adrien Le	93070	residential	DOUBLE	asphalt				limite 30	8		3	32	48.90298234986424, 2.3	
11	'coordinates'	1428600336.0	18e Arrondissement		75118	unclassified	UNIQUE	asphalt			DSC		7		3	34	48.90132258986415, 2.3	
12	'coordinates'	29105023.0	18e Arrondissement	Avenue de la F	75118	primary	UNIQUE	asphalt				limite 30	65		3	32	48.90093867074436, 2.34	
13	'coordinates'	1226939261.0	18e Arrondissement	Rue Francis de	75118	residential	UNIQUE	asphalt		bande uni	voie bus uni	limite 30	42		2	22	48.899482619302184, 2.3	

FIGURE 1 – Extrait du fichier CSV des aménagements cyclab IDF

2.1.1 API Vélib' temps réel

L'API officielle Vélib' est accessible via opendata.paris.fr. Elle retourne pour chaque station son nom, ses coordonnées GPS, sa capacité, le nombre de vélos disponibles et le nombre de bornettes libres. Dans notre application, l'API est appelée à chaque calcul d'itinéraire afin d'avoir les données de disponibilité les plus récentes.

2.1.2 OSMnx et OpenStreetMap

OSMnx est une bibliothèque Python qui permet de télécharger automatiquement le réseau routier et cyclable depuis OpenStreetMap. Elle a été utilisée pour obtenir :

- `G_bike` : le réseau complet accessible à vélo (routes + pistes)
- `G_cyclo` : uniquement les pistes cyclables (`cyclway`, `path`, `track`)

2.1.3 Aménagements cyclables IDF

Ce fichier CSV recense l'ensemble des aménagements vélo en Île-de-France avec le type de voie, la commune et les coordonnées GPS. Il a été exploré en début de projet pour comprendre la structure du réseau cyclable régional. OSMnx couvrant déjà ces informations de manière plus complète et directement intégrable en Python, c'est cette source qui a été retenue pour le calcul d'itinéraires.

2.2 Nettoyage et traitement des données

2.2.1 Données Vélib'

- Suppression des stations non opérationnelles (`is_installed = NON`)
- Suppression des lignes avec des coordonnées GPS manquantes
- Conversion des coordonnées en latitude/longitude via pandas

Après nettoyage : **1 506 stations opérationnelles.**

2.2.2 Réseau cyclable OSMnx

Les graphes téléchargés via OSMnx sont sauvegardés au format `.graphml` pour éviter un nouveau téléchargement à chaque lancement de l'application.

2.2.3 Difficulté rencontrée

La principale difficulté a été la couverture incomplète des données OSMnx en banlieue. Certaines communes comme Nanterre ou Bobigny n'ont pas toutes leurs pistes cyclables référencées dans OpenStreetMap. Cela a conduit à concentrer l'analyse sur **Paris intramuros**, où les données sont fiables et complètes.

3 Modélisation en graphe

3.1 Construction du graphe

Le réseau Vélib' est modélisé sous forme de **graphe non orienté** $G = (V, E)$ où :

- V est l'ensemble des **nœuds** — chaque nœud représente une station Vélib'
- E est l'ensemble des **arêtes** — deux stations sont reliées si une piste cyclable passe à moins de **300 mètres** des deux stations

Le seuil de 300 mètres a été choisi car il correspond à une distance raisonnable à pied entre deux stations (environ 3 à 4 minutes de marche). La construction a été réalisée avec la bibliothèque Python `NetworkX`.

3.2 Caractéristiques du graphe

Caractéristique	Valeur
Nombre de nœuds (stations)	1 506
Nombre d'arêtes (connexions)	3 729
Densité du graphe	0,0033
Nombre de composantes connexes	322
Taille de la plus grande composante	1 062 stations (70%)
Stations isolées	262 stations

TABLE 2 – Caractéristiques du graphe Vélib'

3.3 Visualisation avec Gephi

Le graphe a été exporté au format `.gexf` et visualisé avec le logiciel **Gephi**. Les nœuds sont colorés selon leur degré de centralité (du bleu pour les stations peu connectées au rouge pour les stations très connectées) et leur taille est proportionnelle au nombre de connexions.

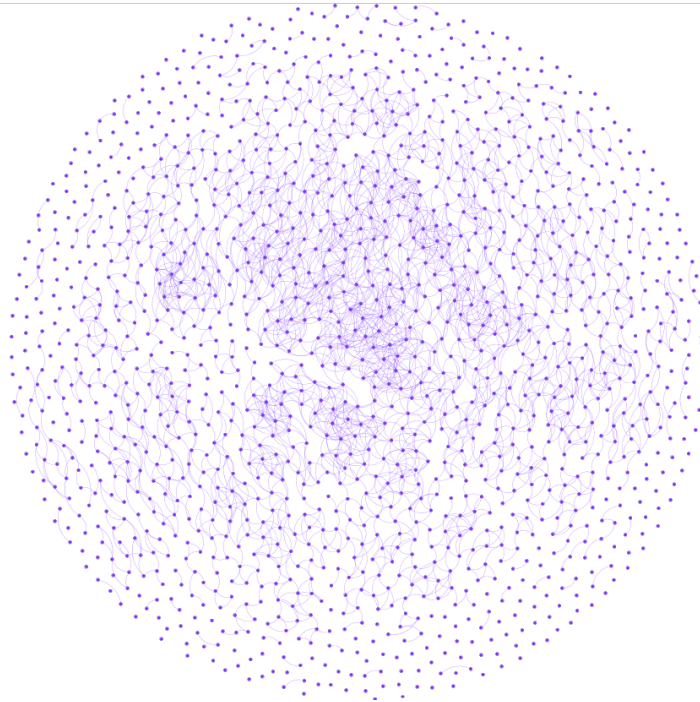


FIGURE 2 – Visualisation du graphe Vélip' avec Gephi — coloré par centralité de degré

On observe sur la figure 2 :

- Un **noyau dense** au centre correspondant à Paris intramuros
- Des **stations isolées** sur les bords correspondant à la banlieue
- Une **fragmentation** importante du réseau entre Paris et la petite couronne

4 Algorithmes appliqués

4.1 Analyse de centralité

La **centralité de degré** a été calculée pour chaque station afin d'identifier les stations les mieux connectées au réseau cyclable. La station avec la centralité la plus élevée est **Caire - Dussoubs** (score : 0,0126), située dans le 2^e arrondissement de Paris.

4.2 Composantes connexes

L'analyse des composantes connexes révèle que le réseau est **fortement fragmenté** : 322 composantes sont identifiées, dont 262 stations complètement isolées. Cette fragmentation est particulièrement marquée en petite couronne où les infrastructures cyclables sont moins développées.

Interprétation : Ces 262 stations isolées représentent des zones où un cycliste ne peut pas relier une station à une autre en restant uniquement sur des pistes cyclables. Cela met en évidence les lacunes du réseau cyclable parisien.

4.3 Algorithme de Dijkstra

4.3.1 Principe

L'algorithme de Dijkstra, publié par Edsger W. Dijkstra en 1959, permet de trouver le **chemin de coût minimal** entre deux nœuds dans un graphe pondéré avec des poids positifs. Sa complexité est $O((V + E) \log V)$.

Dans notre cas avec 1 506 nœuds et 3 729 arêtes, le calcul est quasi instantané.

4.3.2 Deux modes d'itinéraire

Deux modes ont été implémentés en jouant sur les **poids des arêtes** :

Mode	Poids des pistes cyclables	Poids des routes normales
Rapide	distance réelle (m)	distance réelle (m)
Sécurisé	distance $\times$ 0,3 (favorisé)	distance $\times$ 2 (pénalisé)

TABLE 3 – Pondération des arêtes selon le mode de trajet

En mode sécurisé, Dijkstra favorise naturellement les pistes cyclables grâce à leur poids réduit, même si elles allongent légèrement la distance réelle.

4.3.3 Processus complet de calcul

1. Géocodage des adresses via l'API Nominatim (OpenStreetMap)
2. Sélection de la station de départ la plus proche avec des vélos disponibles
3. Sélection de la station d'arrivée la plus proche avec des bornettes libres
4. Application de Dijkstra sur le graphe OSMnx (`nx.shortest_path()`)
5. Segmentation du chemin en tronçons cyclables / non cyclables
6. Calcul du temps total : marche à **5 km/h** + vélo à **15 km/h**
7. Identification des tronçons manquants

La vitesse de 15 km/h correspond à la vitesse moyenne d'un cycliste urbain parisien selon les études de mobilité urbaine.

4.3.4 Identification des tronçons manquants

Pour chaque arête du trajet rapide, l'algorithme vérifie si elle appartient au réseau cyclable OSMnx (attribut `highway = cycleway, path` ou `track`). Les tronçons non cyclables sont identifiés et quantifiés en mètres.

5 Résultats et analyse

5.1 Exemple de trajet

- **Départ** : 1 rue Marx Dormoy, Paris 18^e
- **Arrivée** : 10 avenue Felix Faure, Paris 15^e

Mode	Distance	Temps total	% pistes cyclables
Rapide	5,2 km	32 min	22%
Sécurisé	7,8 km	45 min	74%

TABLE 4 – Comparaison des deux modes de trajet

Observation clé : 13 minutes de différence pour passer de 22% à 74% de pistes cyclables. C’est le coût concret de l’absence d’infrastructures cyclables sur ce trajet.

5.2 Tronçons manquants et coût de sécurisation

Sur ce trajet, **7,2 km** ne sont pas couverts par des pistes cyclables.
Selon le **Club des Villes et Territoires Cyclables et Marchables (CVTCM, 2024)**, le coût moyen d’une piste cyclable en France est de **590 000 EUR/km** en milieu urbain.

$$7,2 \text{ km} \times 590\,000 \text{ EUR/km} = \mathbf{4\,248\,000 \text{ EUR}} \tag{1}$$

4,25 millions d’euros — c’est le montant que la ville de Paris devrait investir pour rendre ce trajet rapide aussi sûr que le trajet sécurisé. Ce chiffre soulève une vraie question politique : est-ce qu’on devrait laisser ce choix au cycliste ?

6 Interface web interactive

6.1 Architecture technique

L’interface web a été développée avec le framework Python **Flask**. La carte interactive utilise la bibliothèque JavaScript **Leaflet.js**.

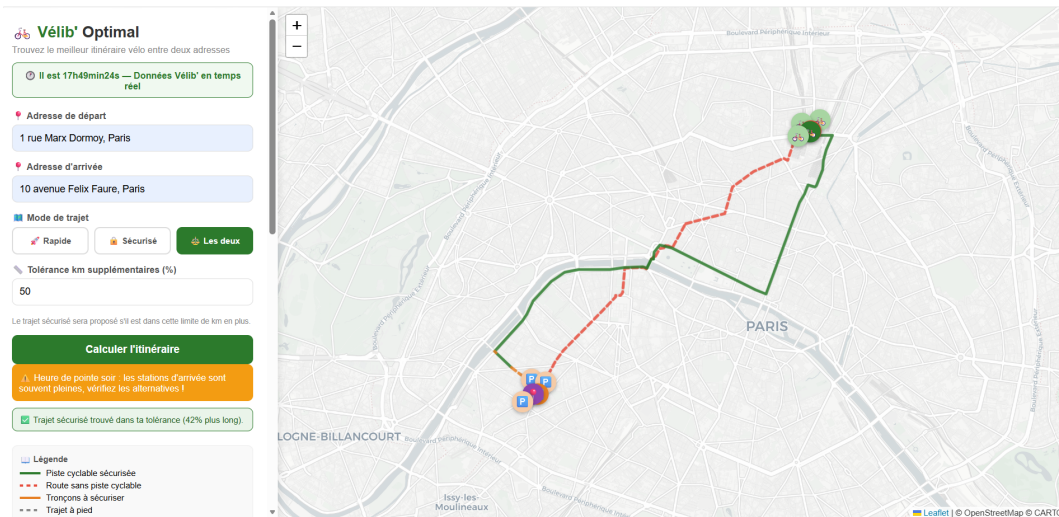


FIGURE 3 – Interface web — Vélib' Optimal avec les deux trajets affichés

6.2 Fonctionnalités principales

- **Horloge temps réel :** l’heure s’affiche en direct, confirmant que les données Vélib’ sont récupérées à l’instant T

- **Deux modes de trajet** : Rapide, Sécurisé, ou Les deux simultanément
- **Tolérance configurable** : l'utilisateur définit le pourcentage de km supplémentaires acceptés pour le mode sécurisé
- **Segments visuels** : trait plein = piste cyclable, pointillés = route sans piste
- **Score de fiabilité** : Fiable / Très vide / Presque pleine
- **3 stations alternatives** : proposées en cas d'indisponibilité
- **Alerte saturation** : si moins de 20% des bornettes sont libres
- **Tronçons manquants** : affichage en orange des rues sans piste cyclable

7 Difficultés rencontrées

Ce projet a présenté plusieurs difficultés techniques et méthodologiques :

1. **Données OSMnx incomplètes en banlieue** : Le réseau cyclable de certaines communes comme Nanterre ou Bobigny est mal référencé dans OpenStreetMap. Des tests initiaux ont montré des itinéraires aberrants avec de grands détours. La décision de se concentrer sur Paris intramuros a résolu ce problème.
2. **Temps de téléchargement des graphes** : Le téléchargement du réseau OSMnx prenait 5 à 10 minutes. La solution a été de sauvegarder les graphes au format `.graphml`.
3. **Passage du CSV statique à l'API temps réel** : Le fichier CSV initial représentait un instantané figé. Le passage à l'API a nécessité de gérer la pagination (requêtes par lots de 100 stations) et les erreurs de connexion.
4. **Types de données non homogènes dans OSMnx** : L'attribut `highway` des arêtes OSMnx peut être une chaîne de caractères ou une liste. Cette incohérence a causé des erreurs de type (`unhashable type: list`) qui ont nécessité une gestion explicite dans le code.
5. **Choix des poids pour le mode sécurisé** : Déterminer les valeurs optimales ($\times 0,3$ et $\times 2$) a nécessité plusieurs tests pour obtenir un résultat réaliste.

8 Limites et perspectives

8.1 Limites identifiées

1. **Données OSMnx incomplètes** : Les pistes cyclables récentes ne sont pas toujours référencées dans OpenStreetMap, notamment en banlieue.
2. **Disponibilité non prédictive** : L'API Vélib' donne la disponibilité au moment T mais ne peut pas anticiper la disponibilité future.
3. **Vitesse moyenne fixe** : La vitesse de 15 km/h est identique pour tous les cyclistes.
4. **Facteur nuit non pris en compte** : Le mode sécurisé ne tient pas compte de l'éclairage public ou de la fréquentation nocturne.
5. **Application locale** : L'application tourne en local et n'est pas accessible au grand public.

8.2 Perspectives d'amélioration

- **Intégration du facteur nuit** : Croiser avec les données d'éclairage public pour un trajet sûr à toute heure.
- **Prédiction de disponibilité** : Historique sur plusieurs semaines pour prédire les périodes de forte demande.
- **Déploiement public** : Héberger sur un serveur cloud.
- **Extension géographique** : Améliorer la couverture en banlieue.

9 Conclusion

Ce projet avait pour ambition d'appliquer la théorie des graphes à un problème urbain concret : l'optimisation des itinéraires Vélib' en Île-de-France. Au terme de ce travail, les résultats obtenus dépassent le simple exercice académique et soulèvent des questions profondes sur la mobilité cyclable à Paris.

Ce que la modélisation en graphe révèle

En représentant chaque station Vélib' comme un nœud et chaque connexion cyclable comme une arête, on découvre une réalité que les données brutes ne montrent pas : **262 stations complètement isolées** du réseau, sans aucune piste cyclable à moins de 300 mètres. Ces stations existent, elles ont des vélos disponibles, mais elles sont déconnectées du réseau cyclable. Un cycliste qui s'y rend est condamné à emprunter des routes partagées avec les voitures.

Cette fragmentation est particulièrement marquée en banlieue — Nanterre, Bobigny, Créteil — où le réseau cyclable est bien moins développé qu'à Paris intramuros. Les **322 composantes connexes** identifiées témoignent d'un réseau encore très morcelé, loin d'un maillage continu et cohérent. La grande composante, qui regroupe 1 062 stations soit 70% du réseau, est essentiellement concentrée à Paris.

Ce que Dijkstra quantifie

L'application de l'algorithme de Dijkstra en deux modes — rapide et sécurisé — a permis de mettre en chiffres un dilemme quotidien que vivent des milliers de cyclistes parisiens sans parfois en avoir conscience.

Sur notre trajet exemple entre la rue Marx Dormoy (18^e) et l'avenue Felix Faure (15^e), le résultat est éclairant :

- Le trajet **rapide** prend **32 minutes** mais passe à seulement **22%** de son parcours sur des pistes cyclables. Le cycliste partage la route avec les voitures sur près de 80% du trajet.
- Le trajet **sécurisé** prend **45 minutes** mais atteint **74%** de pistes cyclables. Il est nettement plus sûr mais coûte **13 minutes supplémentaires**.

Ces 13 minutes de différence représentent le **coût humain** de l'absence d'infrastructures cyclables. C'est le temps que chaque cycliste parisien paie quotidiennement pour avoir le choix entre rapidité et sécurité — un choix qui ne devrait pas exister dans une ville qui se veut favorable au vélo.

Ce que le calcul des tronçons manquants chiffre

Sur ce même trajet, notre algorithme identifie **7,2 km** de routes sans piste cyclable. En appliquant le coût moyen national de **590 000 EUR par kilomètre** selon le Club des Villes et Territoires Cyclables et Marchables (CVTCM, 2024), on obtient un investissement nécessaire de **4,25 millions d'euros** pour sécuriser ce seul trajet.

Ce chiffre est à la fois considérable et relatif. Considérable car il s'agit d'un seul itinéraire parmi des milliers possibles dans Paris. Relatif car, pour mettre les choses en perspective, 1 kilomètre d'autoroute coûte en moyenne 6,2 millions d'euros — soit dix fois plus cher qu'une piste cyclable. Ce projet soulève donc une vraie question de priorité dans les investissements d'infrastructure : **est-ce qu'on devrait laisser ce choix au cycliste ?**

Ce que l'application web apporte

Au-delà de l'analyse théorique, ce projet a abouti à la création d'un **prototype d'application web interactive** qui rend ces résultats accessibles à n'importe quel utilisateur. Grâce à l'intégration de l'API Vélib' en temps réel, l'application affiche à chaque instant la disponibilité exacte des stations et calcule l'itinéraire optimal selon les préférences de l'utilisateur.

L'affichage des segments en vert plein (pistes cyclables) et en pointillés rouges (routes sans piste) permet de visualiser instantanément la qualité du trajet proposé. L'utilisateur peut ainsi faire un choix éclairé et conscient entre rapidité et sécurité.

Les perspectives ouvertes

Ce travail ouvre plusieurs pistes d'amélioration futures qui pourraient enrichir considérablement le projet.

La plus immédiate et la plus significative serait l'**intégration du facteur nuit**. La sécurité d'un cycliste ne dépend pas uniquement de la présence d'une piste cyclable : l'éclairage public, la fréquentation des rues, la visibilité aux intersections sont autant de facteurs qui varient considérablement entre le jour et la nuit. En croisant les données de tronçons cyclables avec les données d'éclairage public disponibles en open data à Paris, il serait possible de proposer un itinéraire véritablement sûr à toute heure.

Une deuxième amélioration porterait sur la **prédiction de disponibilité**. L'API Vélib' donne la disponibilité au moment T, mais un historique collecté sur plusieurs semaines permettrait de prédire les stations qui seront vides aux heures de pointe et de proposer des alternatives plus fiables.

Enfin, le **déploiement public** de l'application sur un serveur cloud permettrait de la rendre accessible à l'ensemble des cyclistes parisiens, transformant ce projet académique en outil utile au quotidien.

Réflexion finale

Ce projet illustre parfaitement la puissance de la théorie des graphes appliquée à des problèmes réels. En modélisant un réseau de transport sous forme de graphe, on peut révéler des structures cachées, quantifier des inégalités, identifier des manques et chiffrer des solutions. La théorie des graphes n'est pas qu'un outil mathématique abstrait — c'est un outil de compréhension et d'action sur le monde réel.

Rendre visible un problème urbain réel — et en chiffrer la solution.

10 Bibliographie

Références

- [1] Hagberg, A., Swart, P., & Chult, D. (2008). *Exploring network structure, dynamics, and function using NetworkX*. Los Alamos National Laboratory.
- [2] Boeing, G. (2017). *OSMnx : New methods for acquiring, constructing, analyzing, and visualizing complex street networks*. Computers, Environment and Urban Systems, 65, 126–139.
- [3] Dijkstra, E. W. (1959). *A note on two problems in connexion with graphs*. Numerische Mathematik, 1(1), 269–271.
- [4] Club des Villes et Territoires Cyclables et Marchables (2024). *Coûts de référence des aménagements cyclables en France*. CVTCM.
- [5] Ville de Paris. *Vélib' — Disponibilité en temps réel*. opendata.paris.fr
- [6] Île-de-France Mobilités. *Aménagements cyclables en Île-de-France*. data.iledefrance-mobilites.fr

11 Webographie

Références

- [1] GitHub — Code source. github.com/ornelamt/ProjetGrapheOpenData
- [2] OpenStreetMap. www.openstreetmap.org
- [3] OpenData Paris. opendata.paris.fr
- [4] Île-de-France Mobilités Open Data. data.iledefrance-mobilites.fr
- [5] Documentation NetworkX. networkx.org
- [6] Documentation OSMnx. osmnx.readthedocs.io
- [7] Gephi. gephi.org
- [8] Flask. flask.palletsprojects.com
- [9] Leaflet.js. leafletjs.com
- [10] Nominatim. nominatim.openstreetmap.org

A Annexe : Code Python — Calcul du chemin sécurisé

```
1 def calculer_chemin_securise(G, noeud_depart, noeud_arrivee):
2     for u, v, k, data in G.edges(keys=True, data=True):
3         highway = data.get("highway", "")
4         if isinstance(highway, list):
5             highway = highway[0]
6         if highway in ["cycleway", "path", "track"]:
7             data["length_secure"] = data["length"] * 0.3
8         else:
9             data["length_secure"] = data["length"] * 2
10    chemin = nx.shortest_path(G, noeud_depart, noeud_arrivee,
11                             weight="length_secure")
12    return chemin
```

Listing 1 – Fonction de calcul du chemin securise avec ponderation